Materiał 03 - PostgreSQL, migracje, relacje i transakcje

Cel

Do tej pory dane znikały po każdym restarcie servera. W tym materiale podłączymy prawdziwą bazę danych **PostgreSQL** i zobaczymy, że "zapisać dane do bazy" to dopiero początek.

Po jego przejściu powinieneś rozumieć:

- jak uruchomić PostgreSQL w kontenerze **Docker**,
- czym są **primary key**, **foreign key**, **constraints** i relacje **1:N** oraz **M:N**,
- czym jest **query builder** i czym różni się od ORM i czystego SQL,
- czym są **migracje** i dlaczego nie tworzymy tabel ręcznie,
- czym jest **connection pool**,
- jak podmienić repository bez zmiany logiki biznesowej,
- czym jest **problem N+1** i jak go uniknąć dzięki `JOIN`,
- jak zrobić **pagination**, filtrowanie i bezpieczne sortowanie,
- czym jest **transakcja** i co oznacza **ACID**,
- czym jest **race condition** i dlaczego `if (count < limit)` w kodzie **nie wystarcza**,
- jak użyć blokady `SELECT ... FOR UPDATE` i constraints bazy jako ostatniej linii obrony.

Kontynuujemy aplikację Task Manager z materiału 02.

> **Wersje użyte w materiale:** Node.js 22, PostgreSQL 16, `knex@3`, `pg@8`. Kod był uruchamiany i testowany na tych wersjach.

---

1. Dlaczego baza danych, a nie plik JSON?

Najprostszy pomysł na "trwałe dane" to zapisywanie tablicy do pliku `data.json`. Działa, dopóki:

- dwa requesty nie zapiszą pliku w tym samym momencie (jeden nadpisze zmiany drugiego),

- server nie padnie w połowie zapisu (plik zostaje uszkodzony),
- danych nie jest tyle, że wczytanie całego pliku, żeby znaleźć jeden rekord, zaczyna trwać sekundy,
- nie potrzebujesz zapytania typu "wszystkie zadania z projektów aktywnych, posortowane po deadline".

System zarządzania bazą danych (DBMS), taki jak PostgreSQL, rozwiązuje te problemy:

- obsługuje wielu klientów jednocześnie,
- gwarantuje, że operacja wykona się **w całości albo wcale**,
- używa indeksów do szybkiego wyszukiwania,
- pilnuje **reguł integralności** danych (np. zadanie nie może wskazywać na nieistniejący projekt).

Dlatego regulamin projektu wymaga bazy działającej jako **osobna usługa** (PostgreSQL, MySQL...), a nie SQLite czy bazy in-memory.

---

2. PostgreSQL w Dockerze

Instalowanie PostgreSQL bezpośrednio w systemie działa, ale każdy ma wtedy inną wersję i konfigurację. Zamiast tego uruchomimy bazę w **kontenerze**.

Kontener to odizolowany proces z własnym systemem plików, zbudowany z gotowego **obrazu** (*image*). Obraz `postgres:16` zawiera kompletną, skonfigurowaną bazę PostgreSQL 16.

Wymagany jest zainstalowany Docker Desktop (Windows/macOS) albo Docker Engine (Linux).

```
docker run --name taskmanager-db \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=postgres \
  -e POSTGRES_DB=taskmanager \
  -p 5432:5432 \
  -v taskmanager-pgdata:/var/lib/postgresql/data \
  -d postgres:16
```

Co oznaczają poszczególne opcje:

Opcja	Znaczenie
`--name taskmanager-db`	nazwa kontenera (żeby łatwo go zatrzymać / uruchomić)
`-e POSTGRES_...`	zmienne środowiskowe - obraz używa ich przy **pierwszym** starcie do utworzenia użytkownika i bazy
`-p 5432:5432`	port komputera `5432` → port kontenera `5432`
`-v taskmanager-pgdata:/var/lib/...`	**volume** - dane bazy przetrwają usunięcie kontenera
`-d`	uruchom w tle (*detached*)
`postgres:16`	obraz i jego wersja

Ważne

Zawsze podawaj **konkretną wersję** obrazu (`postgres:16`), a nie `postgres` / `postgres:latest`. `latest` za pół roku będzie inną wersją, a nowe wersje główne potrafią zmieniać sposób przechowywania danych.

Przydatne komendy:

```
docker ps                          # działające kontenery
docker stop taskmanager-db         # zatrzymanie
docker start taskmanager-db        # ponowne uruchomienie (dane zostają)
docker logs taskmanager-db         # logi bazy

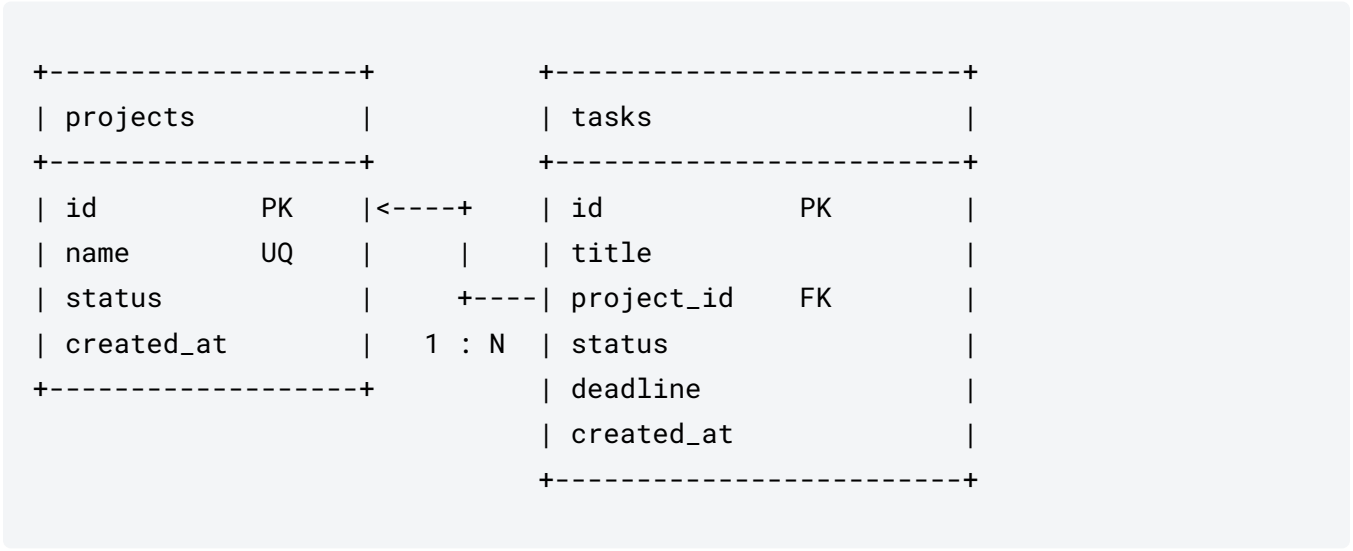
# konsola SQL wewnątrz kontenera
docker exec -it taskmanager-db psql -U postgres -d taskmanager
```

W konsoli `psql` przydadzą się: `\dt` (lista tabel), `\d tasks` (struktura tabeli), `\q` (wyjście).

3. Model relacyjny

W bazie relacyjnej dane przechowujemy w **tabelach**. Każdy wiersz ma **primary key** (PK) - unikalny identyfikator.

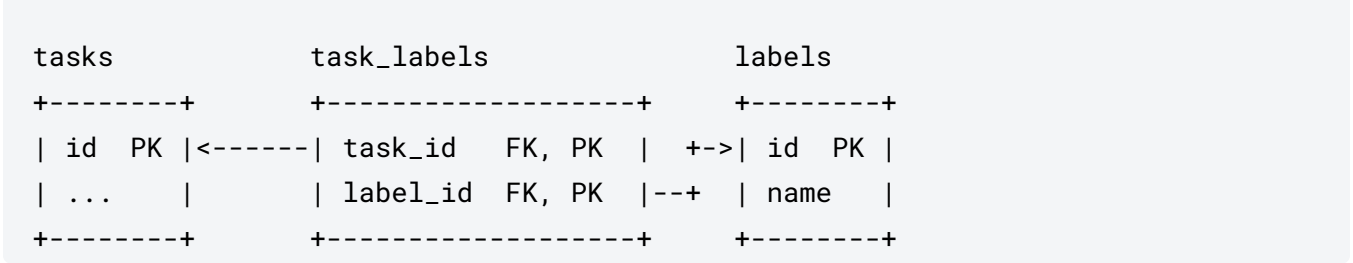
Relacje między tabelami wyrażamy przez **foreign key** (FK) - kolumnę, która wskazuje na PK innej tabeli.



To jest relacja **1:N** (*one-to-many*): jeden projekt ma wiele zadań, każde zadanie należy do dokładnie jednego projektu. FK zawsze leży po stronie "N".

Relacja M:N

Co, jeżeli zadanie może mieć wiele etykiet (*labels*), a etykieta może być przypięta do wielu zadań? Tego nie da się wyrazić jedną kolumną FK. Potrzebna jest **tabela łącząca** (*junction table*):



Para `(task_id, label_id)` jest kluczem głównym tabeli łączącej - ta sama etykieta nie może być przypięta do zadania dwa razy.

Constraints - reguły pilnowane przez bazę

Constraint	Przykład	Co gwarantuje
`NOT NULL`	`title NOT NULL`	wartość musi istnieć
`UNIQUE`	`projects.name`	brak duplikatów
`FOREIGN KEY`	`tasks.project_id → projects.id`	nie da się wskazać nieistniejącego projektu
`CHECK`	`status IN ('TODO', 'IN_PROGRESS', 'DONE')`	wartość spełnia warunek

"Po co, skoro mamy validation w Zod?" - bo **baza nie ufa aplikacji**, tak jak aplikacja nie ufa klientowi. Bug w kodzie, ręczny `UPDATE` w konsoli albo inny serwis podłączony do tej samej bazy - constraint zadziała zawsze. Validation w aplikacji daje czytelne komunikaty błędów, constraints w bazie dają **gwarancję**.

---

4. Jak rozmawiać z bazą z Node.js?

Mamy trzy podejścia:

Podejście	Przykład	Plusy	Minusy
czysty SQL (driver)	`pg`	pełna kontrola, uczysz się SQL	ręczne budowanie dynamicznych zapytań, brak migracji

query builder	**Knex**	SQL pisany w JS, parametryzacja, migracje	nadal musisz rozumieć SQL
ORM	Prisma, Sequelize, TypeORM	mapowanie tabel na obiekty, mniej kodu	"magia", trudniej zobaczyć, jakie SQL powstaje

W tym materiale używamy **Knex**. Jest stabilny, działa z `require` (CommonJS, jak nasz projekt), ma wbudowane migracje, a każde wywołanie bardzo blisko odpowiada konkretnemu SQL - widać, co się dzieje.

```
db("tasks").where({ project_id: 1, status: "TODO" }).orderBy("created_at",
"desc").limit(10);
```

generuje:

```
select * from "tasks" where "project_id" = $1 and "status" = $2 order by
"created_at" desc limit $3
```

Zwróć uwagę na `$1`, `$2`, `$3`. Wartości **nie są wklejane** w tekst zapytania - są wysyłane do bazy osobno (*parameterized query*). Dzięki temu nie da się wstrzyknąć SQL przez dane od użytkownika (**SQL injection**). Wrócimy do tego przy sortowaniu.

W projekcie możesz użyć dowolnego narzędzia (w Spring Boot będzie to JPA/Hibernate, w Django - Django ORM). Koncepcje z tego materiału są takie same.

Instalacja

```
npm install knex pg
```

`knex` to query builder, `pg` to driver PostgreSQL, którego Knex używa pod spodem.

Konfiguracja

Dane do połączenia trzymamy w zmiennej środowiskowej, a nie w kodzie. Tworzymy plik `.env` w głównym katalogu projektu:

```
DATABASE_URL=postgres://postgres:postgres@localhost:5432/taskmanager
```

i **od razu** dodajemy go do `.gitignore`:

```
node_modules
.env
```

Format URL: `postgres://UŻYTKOWNIK:HASŁO@HOST:PORT/BAZA`.

```
// knexfile.js
try {
  process.loadEnvFile();
} catch {
  // brak pliku .env - zmienne mogą przyjść z zewnątrz (np. z Dockera)
}

module.exports = {
  client: "pg",
  connection: process.env.DATABASE_URL,
  migrations: { directory: "./src/db/migrations" },
  seeds: { directory: "./src/db/seeds" }
};
```

`process.loadEnvFile()` to funkcja wbudowana w Node.js (od wersji 20.12 / 21.7). Wczytuje plik `.env` do `process.env`. **Nie nadpisuje** zmiennych, które już są ustawione - dzięki temu zmienne podane z zewnątrz (np. w Dockerze) mają pierwszeństwo. Wcześniej do tego celu używało się

pakietu `dotenv`.

```
// src/db/knex.js
const knex = require("knex");
const config = require("../../knexfile");

module.exports = knex(config);
```

Connection pool

`knex(config)` nie otwiera jednego połączenia. Tworzy **pulę połączeń** (*connection pool*), domyślnie od 2 do 10.

Dlaczego? Nawiązanie połączenia z bazą (TCP, uwierzytelnienie, inicjalizacja sesji) trwa kilka-kilkadziesiąt milisekund. Gdyby każdy request otwierał nowe połączenie, większość czasu traciłby na samo "witanie się". Pula trzyma kilka otwartych połączeń i **wypożycza** je kolejnym zapytaniom.

Konsekwencja: obiekt `db` tworzymy **raz** na całą aplikację (moduł `src/db/knex.js`), a nie w każdym repository.

---

5. Migracje

Problem

Tworzysz tabelę ręcznie w `psql`. Kolega pobiera Twój kod - ale jego baza jest pusta. Za tydzień dodajesz kolumnę `deadline` - musisz pamiętać, żeby dodać ją też na serwerze produkcyjnym. I u kolegi. I w bazie testowej.

Rozwiązanie

Migracja to plik z kodem, który zmienia schemat bazy **o jeden krok**. Migracje są wersjonowane razem z kodem w repozytorium. Każda ma dwie funkcje:

- `up` - wprowadza zmianę,
- `down` - cofa ją.

```
npx knex migrate:make create_projects_and_tasks
```

Knex utworzy plik z timestampem w nazwie, np. `src/db/migrations/20261020120000_create_projects_and_tasks.js`. Timestamp wyznacza **kolejność** wykonywania.

```
// src/db/migrations/20261020120000_create_projects_and_tasks.js
exports.up = async function (knex) {
  await knex.schema.createTable("projects", table => {
    table.increments("id").primary();
    table.string("name", 120).notNullable().unique();
    table.string("status", 20).notNullable().defaultTo("ACTIVE");
    table.timestamp("created_at", { useTz:
true }).notNullable().defaultTo(knex.fn.now());
  });

  await knex.schema.createTable("tasks", table => {
    table.increments("id").primary();
    table.string("title", 120).notNullable();
    table.text("description");
    table.string("status", 20).notNullable().defaultTo("TODO");
    table.integer("project_id").notNullable()
      .references("id").inTable("projects").onDelete("CASCADE");
    table.timestamp("deadline", { useTz: true });
    table.timestamp("created_at", { useTz:
true }).notNullable().defaultTo(knex.fn.now());
    table.index(["project_id", "status"]);
  });

  await knex.raw(`ALTER TABLE projects ADD CONSTRAINT projects_status_check
CHECK (status IN ('ACTIVE', 'ARCHIVED'))`);
  await knex.raw(`ALTER TABLE tasks ADD CONSTRAINT tasks_status_check CHECK
(status IN ('TODO', 'IN_PROGRESS', 'DONE'))`);
};

exports.down = async function (knex) {
```

```
  await knex.schema.dropTable("tasks");
  await knex.schema.dropTable("projects");
};
```

Kilka decyzji wartych wyjaśnienia:

`increments("id")` tworzy kolumnę `SERIAL` - baza sama nadaje kolejne ID. Koniec z licznikiem w kodzie.

`timestamp(..., { useTz: true })` tworzy kolumnę `timestamptz`. PostgreSQL zapisuje wtedy moment w czasie w UTC i poprawnie przelicza strefy. Bez `useTz` zapisuje "datę z zegara" bez informacji o strefie - źródło wielu błędów przy zmianie czasu letniego / zimowego.

`onDelete("CASCADE")` - usunięcie projektu usunie jego zadania. Alternatywy: `RESTRICT` (zabroń usunięcia projektu, który ma zadania), `SET NULL` (zostaw zadania bez projektu). Wybór to **decyzja biznesowa**, nie techniczna.

`index(["project_id", "status"])` - bez indeksu zapytanie "ile zadań `IN_PROGRESS` ma projekt 1" przeszukuje **całą** tabelę. Z indeksem baza od razu trafia do pasujących wierszy. Indeks zakładamy pod zapytania, które naprawdę wykonujemy.

`knex.raw(...)` **dla CHECK** - nie każda funkcja SQL ma odpowiednik w API Knexa. `knex.raw` pozwala napisać fragment SQL bezpośrednio.

`down` **usuwa tabele w odwrotnej kolejności** - `tasks` ma FK do `projects`, więc najpierw trzeba usunąć `tasks`.

Uruchamianie

```
npx knex migrate:latest     # wykonaj wszystkie niewykonane migracje
npx knex migrate:status     # które migracje są wykonane
npx knex migrate:rollback   # cofnij ostatnią "paczkę" (batch) migracji
```

Skąd Knex wie, które migracje są wykonane? Tworzy w bazie tabelę `knex_migrations` i zapisuje w niej nazwę każdej wykonanej migracji.

Ważne

Nigdy nie edytuj migracji, która została już wykonana (u Ciebie, u kolegi, na produkcji). Knex jej drugi raz nie uruchomi - bo według `knex_migrations` jest już wykonana. Zmiana schematu = **nowa** migracja (`migrate:make add_deadline_to_tasks`).

Wyjątek: migracja, która istnieje tylko na Twoim komputerze i nikt jej jeszcze nie pobrał. Wtedy `rollback` → edycja → `latest`.

---

6. Seeds - dane startowe

Seed to skrypt wypełniający bazę przykładowymi danymi (do developmentu i demo).

```
npx knex seed:make 01_projects_and_tasks
```

```
// src/db/seeds/01_projects_and_tasks.js
exports.seed = async function (knex) {
  await knex.raw("TRUNCATE tasks, projects RESTART IDENTITY CASCADE");

  await knex("projects").insert([
    { name: "Website", status: "ACTIVE" },
    { name: "Old CRM", status: "ARCHIVED" }
  ]);

  await knex("tasks").insert([
    { title: "Landing page", project_id: 1, status: "IN_PROGRESS" },
    { title: "Contact form", project_id: 1, status: "IN_PROGRESS" },
    { title: "Pricing page", project_id: 1, status: "TODO" },
    { title: "Blog", project_id: 1, status: "TODO" }
  ]);
};
```

```
npx knex seed:run
```

`TRUNCATE ... RESTART IDENTITY` czyści tabele i resetuje liczniki ID - po każdym `seed:run` dostajesz identyczny stan. Seedy uruchamiane są w kolejności alfabetycznej nazw plików, stąd prefiks `01_`.

> Migracje = **struktura** (zawsze, na każdym środowisku). Seeds = **przykładowe dane** (tylko development / demo). Nigdy nie uruchamiaj seeda z `TRUNCATE` na produkcji.

---

7. Repository z Knex

Teraz najważniejszy moment. Piszemy **nową implementację** tego samego interfejsu repository, który zdefiniowaliśmy w materiale 02.

```
// src/modules/tasks/tasks.repository.knex.js
const SORT_COLUMNS = { createdAt: "created_at", deadline: "deadline", title:
"title" };

function toTask(row) {
  if (!row) return null;
  return {
    id: row.id,
    title: row.title,
    description: row.description,
    status: row.status,
    projectId: row.project_id,
    deadline: row.deadline,
    createdAt: row.created_at
  };
}

function createKnexRepositories(db) {
  const projects = {
    async findById(id) {
      return (await db("projects").where({ id }).first()) ?? null;
```

```
    },
    async lockById(id) {
      await db("projects").where({ id }).forUpdate().first();
    }
  };

  const tasks = {
    async findById(id) {
      return toTask(await db("tasks").where({ id }).first());
    },

    async create({ title, description, projectId, deadline, status }) {
      const [row] = await db("tasks")
        .insert({ title, description, project_id: projectId, deadline,
status })
        .returning("*");
      return toTask(row);
    },

    async updateStatus(id, status) {
      const [row] = await
db("tasks").where({ id }).update({ status }).returning("*");
      return toTask(row);
    },

    async countByProjectAndStatus(projectId, status) {
      const { count } = await db("tasks")
        .where({ project_id: projectId, status })
        .count({ count: "*" })
        .first();
      return Number(count);
    },

    async list({ status, projectId, page, limit, sort }) {
      const base = db("tasks").modify(qb => {
        if (status) qb.where("status", status);
        if (projectId) qb.where("project_id", projectId);
      });

      const desc = sort.startsWith("-");
      const column = SORT_COLUMNS[desc ? sort.slice(1) : sort];

      const [rows, { count }] = await Promise.all([
```

```
        base.clone()
          .select("*")
          .orderBy(column, desc ? "desc" : "asc")
          .orderBy("id", "asc")
          .limit(limit)
          .offset((page - 1) * limit),
        base.clone().count({ count: "*" }).first()
      ]);

      return { items: rows.map(toTask), total: Number(count) };
    }
  };

  return { projects, tasks };
}

module.exports = { createKnexRepositories };
```

Omówmy najważniejsze elementy:

`toTask(row)` - **mapowanie** `snake_case` → `camelCase`. W SQL konwencją jest `project_id`, w JavaScript `projectId`. Repository jest jedynym miejscem, które zna nazwy kolumn. Service i controller widzą wyłącznie `projectId`.

`.first()` - zwraca pierwszy wiersz albo `undefined`, zamiast tablicy.

`.returning("*")` - PostgreSQL potrafi w jednym zapytaniu wstawić wiersz i zwrócić go (razem z ID i wartościami domyślnymi nadanymi przez bazę). Bez tego potrzebne byłoby drugie zapytanie.

`Number(count)` - `COUNT(*)` w PostgreSQL zwraca typ `bigint` (64 bity). JavaScript `number` bezpiecznie reprezentuje liczby całkowite tylko do 2^53 - 1, więc driver `pg` zwraca `bigint` jako **string** (`"4"`, a nie `4`). Dla liczników możemy bezpiecznie zamienić go na liczbę. Bez `Number()` porównanie `count >= 3` działałoby przypadkiem (JavaScript skonwertuje string), ale `count + 1` dałoby `"41"`.

`.modify(qb => ...)` - pozwala dynamicznie dokładać warunki. Filtr dodajemy tylko, gdy został podany.

`.clone()` - z tej samej bazy zapytania robimy dwa: jedno pobiera stronę danych, drugie liczy wszystkie pasujące wiersze. Bez `clone()` oba modyfikowałyby ten sam obiekt.

`Promise.all` - oba zapytania są niezależne, więc wysyłamy je równolegle (każde dostaje osobne połączenie z puli).

---

8. Pagination, filtrowanie i sortowanie

Endpoint `GET /api/tasks` bez ograniczeń przy 100 000 zadań zwróciłby 100 000 obiektów. Klient tego nie potrzebuje, a server i sieć by to odczuły.

Offset pagination

```
GET /api/tasks?page=2&limit=20
```

```
SELECT ... ORDER BY created_at DESC, id ASC LIMIT 20 OFFSET 20
```

Response zawiera dane **i metadane**:

```
{
  "data": [ ... ],
  "meta": { "page": 2, "limit": 20, "total": 134, "totalPages": 7 }
}
```

Dzięki `total` frontend może narysować numerację stron.

Schemat query rozbudowujemy o parametry z wartościami domyślnymi:

```
```

```
// src/modules/tasks/tasks.schemas.js (fragment)
const listTasksQuery = z.object({
  status: z.enum(["TODO", "IN_PROGRESS", "DONE"]).optional(),
  projectId: z.coerce.number().int().positive().optional(),
  page: z.coerce.number().int().min(1).default(1),
  limit: z.coerce.number().int().min(1).max(100).default(20),
  sort: z.enum(["createdAt", "-createdAt", "deadline", "-deadline", "title",
"-title"]).default("-createdAt")
});
```

`max(100)` - bez górnego limitu klient może zażądać `limit=1000000` i obejść całą ideę pagination.

`sort` - konwencja: `title` = rosnąco, `-title` = malejąco.

Ważne - dlaczego zawsze `ORDER BY`?

Bez `ORDER BY` baza **nie gwarantuje** żadnej kolejności. Ten sam `LIMIT 20 OFFSET 20` może zwrócić inne wiersze przy kolejnym wywołaniu - niektóre zadania pojawią się na dwóch stronach, innych nie zobaczysz wcale.

Dodatkowe `.orderBy("id", "asc")` to tzw. **tie-breaker**: jeżeli dwa zadania mają identyczne `created_at`, kolejność i tak jest jednoznaczna.

Ważne - SQL injection przy sortowaniu

Wartości w `where` są parametryzowane. Ale **nazwy kolumn** w `orderBy` nie mogą być parametrem - trafiają do tekstu zapytania. Dlatego nigdy nie robimy:

```
// NIE RÓB TEGO
.orderBy(req.query.sort)
```

Używamy **whitelisty**: schemat Zod akceptuje tylko konkretne wartości (`z.enum`), a mapa `SORT_COLUMNS` tłumaczy je na nazwy kolumn. Klient nie ma żadnej możliwości przekazania własnego fragmentu SQL.

Wada offset pagination

`OFFSET 100000` oznacza, że baza musi przejść i odrzucić 100 000 wierszy, zanim zwróci kolejne 20. Przy bardzo dużych tabelach stosuje się **keyset (cursor) pagination**: `WHERE created_at < :ostatnia_wartość ORDER BY created_at DESC LIMIT 20`. Dla projektu na zaliczenie offset w zupełności wystarcza.

---

9. JOIN i problem N+1

Frontend chce listę zadań z **nazwą projektu**. Pierwszy pomysł:

```
// NIE RÓB TEGO
const tasks = await db("tasks").limit(20);
for (const task of tasks) {
  const project = await db("projects").where({ id: task.project_id }).first();
  task.projectName = project.name;
}
```

Dla 20 zadań to **21 zapytań**: jedno po listę + jedno na każde zadanie. Stąd nazwa: **N+1**. Każde zapytanie to podróż do bazy i z powrotem. Przy 100 elementach na stronie i opóźnieniu 2 ms na zapytanie tracisz 200 ms tylko na komunikację.

Rozwiązanie - **jedno** zapytanie z `JOIN`:

```
const rows = await db("tasks as t")
  .join("projects as p", "p.id", "t.project_id")
  .select("t.id", "t.title", "t.status", "p.id as projectId", "p.name as
projectName")
  .orderBy("t.created_at", "desc")
  .limit(20);
```

```
select "t"."id", "t"."title", "t"."status", "p"."id" as "projectId",
"p"."name" as "projectName"
from "tasks" as "t" inner join "projects" as "p" on "p"."id" =
"t"."project_id"
order by "t"."created_at" desc limit $1
```

Rodzaje JOIN-ów, które przydadzą się najczęściej:

- `join` (`INNER JOIN`) - tylko wiersze, które mają dopasowanie po obu stronach,
- `leftJoin` (`LEFT JOIN`) - wszystkie wiersze z lewej tabeli; jeżeli brak dopasowania, kolumny z prawej są `NULL` (np. projekty **razem z tymi**, które nie mają zadań).

Jak zobaczyć, jakie zapytania wykonuje aplikacja?

```
DEBUG=knex:query node src/server.js
```

(Windows PowerShell: `$env:DEBUG="knex:query"; node src/server.js`)

Każde zapytanie SQL zostanie wypisane w konsoli. To najprostszy sposób, żeby wykryć N+1: jeżeli jeden request generuje dziesiątki niemal identycznych zapytań - masz problem.

---

10. Podmiana repository - payoff architektury z materiału 02

W materiale 02 service dostawał repositories jako parametry. Teraz w `server.js` podajemy implementację Knex zamiast in-memory - i **reguły biznesowe działają dalej bez zmian**.

Jedną zmianę w service jednak zrobimy. Wynika z czegoś, czego w pamięci nie było: **transakcji**. O tym w kolejnych sekcjach.

---

11. Transakcje i ACID

Problem

Załóżmy operację "przenieś zadanie do innego projektu i zapisz wpis w historii". To dwa zapytania. Co, jeżeli pierwsze się wykona, a drugie nie (np. padnie połączenie)? Zadanie jest przeniesione, ale historia o tym nie wie. Dane są **niespójne**.

Transakcja

Transakcja grupuje kilka operacji w jedną całość:

```
BEGIN;
  UPDATE tasks SET project_id = 2 WHERE id = 5;
  INSERT INTO task_history (...) VALUES (...);
COMMIT;      -- albo ROLLBACK, jeżeli coś poszło nie tak
```

Bazy relacyjne gwarantują dla transakcji właściwości **ACID**:

	Właściwość	Znaczenie
A	Atomicity	wszystko albo nic - przy błędzie wszystkie zmiany są cofane
C	Consistency	po transakcji wszystkie constraints są spełnione
I	Isolation	równoległe transakcje nie widzą swoich niezatwierdzonych zmian
D	Durability	po `COMMIT` dane przetrwają nawet awarię servera

Transakcje w Knex

```
await db.transaction(async trx => {
  await trx("tasks").where({ id: 5 }).update({ project_id: 2 });
  await trx("task_history").insert({ task_id: 5, change: "moved to project
2" });
```

```
});
```

Jeżeli funkcja zakończy się normalnie - Knex robi `COMMIT`. Jeżeli rzuci wyjątek - `ROLLBACK`, a wyjątek leci dalej.

Kluczowy szczegół: wewnątrz transakcji **wszystkie** zapytania muszą używać obiektu `trx`, a nie `db`. Zapytanie przez `db` pobierze **inne** połączenie z puli i wykona się poza transakcją.

Jak przekazać transakcję do repository?

Service nie może znać Knexa (to szczegół warstwy danych). Rozwiązanie: funkcja `runInTransaction`, która uruchamia kod w transakcji i podaje mu **repositories powiązane z tą transakcją**:

```
// src/modules/tasks/tasks.repository.knex.js (dopisujemy na końcu)
function createKnexUnitOfWork(db) {
  return {
    runInTransaction: fn => db.transaction(trx =>
fn(createKnexRepositories(trx)))
  };
}

module.exports = { createKnexRepositories, createKnexUnitOfWork };
```

`createKnexRepositories(trx)` - te same repositories, ale zamiast `db` dostają `trx`. To działa, bo w Knex obiekt transakcji ma to samo API co `db`.

Wersja in-memory po prostu wywołuje funkcję (w pamięci nie ma czego cofać):

```
// src/modules/tasks/tasks.repository.memory.js (zmiana na końcu)
  const repositories = { projects, tasks };
  return { repositories, runInTransaction: fn => fn(repositories) };
```

(W repository in-memory zmieniamy też nazwy na `projects` / `tasks`, dodajemy pustą metodę

`lockById` i pagination w `list` - tak, żeby **interfejs** obu implementacji był identyczny.)

Ten wzorzec nazywa się **Unit of Work**.

---

12. Race condition - dlaczego `if` nie wystarcza

To najważniejsza sekcja tego materiału.

Przypomnij sobie regułę WIP limit:

```
const inProgress = await
tasksRepository.countByProjectAndStatus(task.projectId, "IN_PROGRESS");
if (inProgress >= WIP_LIMIT) throw new ConflictError(...);
return tasksRepository.updateStatus(id, "IN_PROGRESS");
```

Projekt ma 2 zadania `IN_PROGRESS` (limit to 3). Dwóch użytkowników **w tym samym momencie** przenosi do `IN_PROGRESS` dwa różne zadania:

```
czas   Request A (zadanie 3)              Request B (zadanie 4)
----   ---------------------              ---------------------
 t1    COUNT -> 2  (2 < 3, OK)
 t2                                       COUNT -> 2  (2 < 3, OK)
 t3    UPDATE task 3 -> IN_PROGRESS
 t4                                       UPDATE task 4 -> IN_PROGRESS

wynik: 4 zadania IN_PROGRESS, choć limit to 3
```

Oba requesty sprawdziły warunek **zanim** którykolwiek zapisał zmianę. Ten wzorzec nazywa się **check-then-act** i jest klasycznym źródłem **race condition**.

"Przecież Node.js jest jednowątkowy!"

To prawda - JavaScript wykonuje się w jednym wątku. Ale każde `await` na zapytaniu do bazy **oddaje sterowanie**. W czasie, gdy request A czeka na wynik `COUNT`, Node.js obsługuje request B. Do tego w produkcji działa zwykle **kilka instancji** aplikacji (kilka procesów, kilka serwerów) - wszystkie korzystają z jednej bazy.

Czy sama transakcja pomaga?

Nie. Domyślny poziom izolacji w PostgreSQL to **Read Committed**: każde zapytanie widzi dane zatwierdzone przed jego rozpoczęciem. Obie transakcje policzą 2, bo żadna nie widzi niezatwierdzonego `UPDATE` drugiej. Transakcja daje atomowość, ale **nie serializuje** równoległych operacji.

Rozwiązanie 1 - blokada wiersza (`SELECT ... FOR UPDATE`)

```
SELECT * FROM projects WHERE id = 1 FOR UPDATE;
```

`FOR UPDATE` zakłada **blokadę** na wierszu projektu do końca transakcji. Inna transakcja, która spróbuje zablokować ten sam wiersz, **czeka**, aż pierwsza zrobi `COMMIT` lub `ROLLBACK`.

```
czas   Request A                          Request B
----   ---------                          ---------
 t1    BEGIN; LOCK project 1  (OK)
 t2                                       BEGIN; LOCK project 1  -> CZEKA...
 t3    COUNT -> 2, UPDATE task 3
 t4    COMMIT (blokada zwolniona)
 t5                                       ...dostaje blokadę
 t6                                       COUNT -> 3  -> 409
WIP_LIMIT_EXCEEDED
```

Blokujemy **projekt**, a nie zadania, bo reguła dotyczy projektu jako całości: "w tym projekcie może być max 3 zadania `IN_PROGRESS`". Wiersz projektu działa jak "zamek" dla tej reguły.

Service po zmianach:

```
// src/modules/tasks/tasks.service.js
const { NotFoundError, ConflictError, ValidationError } = require("../../
errors/AppError");

const WIP_LIMIT = 3;
const ALLOWED_TRANSITIONS = {
  TODO: ["IN_PROGRESS"],
  IN_PROGRESS: ["DONE", "TODO"],
  DONE: []
};

function createTasksService({ runInTransaction, now = () => new Date() }) {
  async function createTask(input) {
    return runInTransaction(async ({ projects, tasks }) => {
      const project = await projects.findById(input.projectId);
      if (!project) throw new NotFoundError("Project");
      if (project.status === "ARCHIVED") {
        throw new ConflictError("Cannot add tasks to an archived project",
"PROJECT_ARCHIVED");
      }
      if (input.deadline && new Date(input.deadline) <= now()) {
        throw new ValidationError([{ location: "body", field: "deadline",
message: "Deadline must be in the future" }]);
      }
      return tasks.create({ ...input, status: "TODO" });
    });
  }

  async function changeStatus(id, newStatus) {
    return runInTransaction(async ({ projects, tasks }) => {
      const task = await tasks.findById(id);
      if (!task) throw new NotFoundError("Task");

      if (task.status === newStatus) return task;

      if (!ALLOWED_TRANSITIONS[task.status].includes(newStatus)) {
        throw new ConflictError(`Cannot change status from ${task.status} to $
{newStatus}`, "INVALID_STATUS_TRANSITION");
      }

      if (newStatus === "IN_PROGRESS") {
        await projects.lockById(task.projectId);
```

```
        const inProgress = await tasks.countByProjectAndStatus(task.projectId,
"IN_PROGRESS");
        if (inProgress >= WIP_LIMIT) {
          throw new ConflictError(`Project already has ${WIP_LIMIT} tasks in
progress`, "WIP_LIMIT_EXCEEDED");
        }
      }
      return tasks.updateStatus(id, newStatus);
    });
  }

  async function getTask(id) {
    return runInTransaction(async ({ tasks }) => {
      const task = await tasks.findById(id);
      if (!task) throw new NotFoundError("Task");
      return task;
    });
  }

  async function listTasks(query) {
    return runInTransaction(async ({ tasks }) => {
      const { items, total } = await tasks.list(query);
      return {
        items,
        meta: { page: query.page, limit: query.limit, total, totalPages:
Math.ceil(total / query.limit) }
      };
    });
  }

  return { createTask, changeStatus, getTask, listTasks };
}

module.exports = { createTasksService, WIP_LIMIT };
```

Porównaj z wersją z materiału 02: **reguły są identyczne**. Zmieniło się tylko to, że kod działa wewnątrz `runInTransaction`, a przed liczeniem zakładamy blokadę (`projects.lockById`).

> `getTask` i `listTasks` nie potrzebują transakcji (to pojedyncze odczyty). Używamy `runInTransaction` dla spójności interfejsu - service ma jeden sposób dostępu do repositories. Koszt to dodatkowe `BEGIN` / `COMMIT`, przy tej skali pomijalny.

Controller dla listy zwraca teraz dane z metadanymi:

```
// src/modules/tasks/tasks.controller.js (fragment)
async list(req, res) {
  const { items, meta } = await tasksService.listTasks(req.validated.query);
  res.json({ data: items.map(toTaskResponse), meta });
},
```

Rozwiązanie 2 - constraint w bazie

Część reguł da się wyrazić **bezpośrednio jako constraint**. Wtedy baza sama gwarantuje poprawność, niezależnie od tego, ile instancji aplikacji działa i czy ktoś pamiętał o blokadzie.

Przykłady:

- "email użytkownika musi być unikalny" → `UNIQUE (email)`,
- "projekt może mieć tylko jedno zadanie oznaczone jako główne" → **partial unique index**: `CREATE UNIQUE INDEX ... ON tasks (project_id) WHERE is_main = true`,
- "dwa przedziały czasu w tym samym zasobie nie mogą się nakładać" → **exclusion constraint** w PostgreSQL (przyda się w zadaniu).

Exclusion constraint wymaga rozszerzenia `btree_gist` (dostępne w oficjalnym obrazie `postgres`):

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE bookings ADD CONSTRAINT bookings_no_overlap
  EXCLUDE USING gist (
    resource_id WITH =,
    tstzrange(start_time, end_time, '[)') WITH &&
  ) WHERE (status = 'ACTIVE');
```

Czytamy to: "nie mogą istnieć dwa wiersze, dla których `resource_id` jest równe (`=`) **i** przedziały czasu się nakładają (`&&`) - biorąc pod uwagę tylko wiersze ze statusem `ACTIVE`". `tstzrange(..., '[)')` tworzy przedział półotwarty: początek należy do przedziału, koniec nie.

Próba wstawienia nakładającego się wiersza kończy się błędem PostgreSQL z kodem `23P01` (*exclusion_violation*).

Którego rozwiązania użyć?

	`FOR UPDATE`	**Constraint w bazie**
Gdzie żyje reguła	w kodzie aplikacji	w schemacie bazy
Czy działa przy wielu instancjach	tak (blokada jest w bazie)	tak
Co jeśli ktoś zapomni o blokadzie	reguła nie działa	constraint działa zawsze
Jakie reguły da się wyrazić	dowolne	tylko te, które da się zapisać jako constraint

W praktyce często stosuje się **oba**: sprawdzenie w service daje czytelny komunikat błędu, constraint w bazie jest ostatnią linią obrony.

Jak zobaczyć race condition na własne oczy?

Race condition jest trudny do zaobserwowania, bo "okno" między `COUNT` a `UPDATE` trwa milisekundy. Żeby go wymusić:

1. **Tymczasowo** usuń wywołanie `projects.lockById(...)`.
2. **Tymczasowo** dodaj sztuczne opóźnienie między sprawdzeniem a zapisem:

```
const inProgress = await tasks.countByProjectAndStatus(task.projectId,
"IN_PROGRESS");
await new Promise(resolve => setTimeout(resolve, 200)); // TYLKO DO
DEMONSTRACJI
if (inProgress >= WIP_LIMIT) { ... }
```

3. Wyślij równolegle kilka requestów:

```
// scripts/race-demo.js
const BASE = "http://localhost:3000/api/tasks";

async function start(taskId) {
  const res = await fetch(`${BASE}/${taskId}/status`, {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ status: "IN_PROGRESS" })
  });
  return `task ${taskId}: ${res.status}`;
}

(async () => {
  const results = await Promise.all([start(3), start(4)]);
  console.log(results);
})();
```

```
npx knex seed:run && node scripts/race-demo.js
```

`fetch` jest wbudowany w Node.js od wersji 18 - nie trzeba nic instalować.

Bez blokady oba requesty dostaną `200`, a w bazie będą 4 zadania `IN_PROGRESS`. Po przywróceniu `lockById` jeden dostanie `200`, drugi `409` - mimo sztucznego opóźnienia. Na koniec **usuń opóźnienie**.

---

13. Błędy z bazy danych w error handlerze

Gdy zadziała constraint, driver `pg` rzuca błąd z polem `code` zgodnym ze standardem SQLSTATE. Najczęstsze:

Kod	Nazwa	Kiedy	Sensowny status HTTP
`23505`	unique_violation	duplikat wartości `UNIQUE`	`409`
`23503`	foreign_key_violation	FK wskazuje na nieistniejący wiersz	`409` lub `400`
`23514`	check_violation	naruszenie `CHECK`	`400`
`23P01`	exclusion_violation	naruszenie exclusion constraint	`409`

Bez obsługi każdy z nich skończy się `500`, choć to nie jest błąd servera. Dopisujemy w error handlerze (przed obsługą błędów nieoczekiwanych):

```
// src/middleware/errorHandler.js (fragment)
if (err.code === "23505") {
  return res.status(409).json({
    error: { code: "UNIQUE_VIOLATION", message: "Resource already exists",
requestId: req.id }
  });
}
```

Nie przekazujemy klientowi `err.detail` z PostgreSQL - zawiera nazwy tabel, kolumn i wartości.

---

14. Składamy wszystko razem

```
// src/app.js
const express = require("express");
const requestLogger = require("./middleware/requestLogger");
const notFound = require("./middleware/notFound");
const errorHandler = require("./middleware/errorHandler");
const { createTasksService } = require("./modules/tasks/tasks.service");
const { createTasksController } = require("./modules/tasks/tasks.controller");
const { createTasksRouter } = require("./modules/tasks/tasks.routes");
```

```
function createApp({ runInTransaction }) {
  const app = express();
  app.use(requestLogger);
  app.use(express.json());

  const tasksService = createTasksService({ runInTransaction });
  app.use("/api/tasks",
createTasksRouter(createTasksController(tasksService)));

  app.use(notFound);
  app.use(errorHandler);
  return app;
}

module.exports = { createApp };
```

```
// src/server.js
const db = require("./db/knex");
const { createApp } = require("./app");
const { createKnexUnitOfWork } = require("./modules/tasks/
tasks.repository.knex");

const app = createApp(createKnexUnitOfWork(db));

app.listen(3000, () => console.log("Server running on http://
localhost:3000"));
```

Pełna procedura uruchomienia od zera:

```
docker start taskmanager-db     # albo docker run ... (sekcja 2), jeżeli
kontener nie istnieje
npx knex migrate:latest
npx knex seed:run
node src/server.js
```

Teraz zrestartuj server - dane **zostają**.

15. Podsumowanie

```
Request
  -> validate (Zod)
  -> controller
  -> service
       runInTransaction(async repos => {
         lock (FOR UPDATE)
         sprawdzenie reguł
         zapis
       })                          -> COMMIT / ROLLBACK
  -> repository (Knex, trx)
  -> PostgreSQL (constraints = ostatnia linia obrony)
```

Najważniejsze rzeczy:

- **Migracje** to historia zmian schematu w repozytorium. Nie edytujemy wykonanych migracji.
- **Constraints** w bazie gwarantują integralność danych niezależnie od kodu aplikacji.
- **Repository** jest jedynym miejscem, które zna nazwy tabel i kolumn.
- **Pagination** zawsze z `ORDER BY` i tie-breakerem. Sortowanie tylko po kolumnach z whitelisty.
- **N+1** wykrywasz przez logowanie zapytań, naprawiasz przez `JOIN`.
- **Transakcja** daje "wszystko albo nic", ale **nie chroni** przed race condition przy check-then-act.
- Przed race condition chroni **blokada** (`FOR UPDATE`) albo **constraint** w bazie - najlepiej oba.
- Błędy constraints mapujemy na właściwe kody HTTP (`409`, `400`), a nie `500`.

W następnym materiale dodamy użytkowników: rejestrację, logowanie, JWT oraz kontrolę dostępu.